

Tecnicatura en Programación a Distancia - UTN

Trabajo Practico Integrador - Programación 2

Estudiantes

- Darío Frison
- Maximiliano Agüero

Repositorio:

https://github.com/dfrison12/TPI_Prog_II

Video:

<https://www.youtube.com/watch?v=1HPOnDzVsns>

Food Store - Sistema de gestión de pedidos de comida

Índice

1. Introducción -----	1
2. Marco teórico y tecnologías -----	2
3. Diseño y arquitectura -----	2
4. Modelo aplicado en el código -----	3
5. Funcionamiento del sistema -----	4
6. Pruebas realizadas -----	6
7. Dificultades y soluciones -----	7
8. Bibliografía y enlaces -----	7

1. Introducción

Food Store es una aplicación de consola para gestionar pedidos de comida. Permite trabajar con categorías, productos, usuarios y pedidos con sus detalles.

Toda la interacción se hace desde el menú de la consola. No se usa base de datos, porque los datos se guardan en memoria usando colecciones. Por eso, al cerrar el programa, la información cargada durante la ejecución no queda guardada.

El objetivo principal fue armar un CRUD completo, respetando el modelo pedido en la consigna y separando el código en partes con responsabilidades claras.

2. Marco teórico y tecnologías

El proyecto se hizo con Java 21 que nos permite trabajar con clases, objetos, interfaces, enums, colecciones y excepciones.

La programación orientada a objetos se aplica en las entidades principales del sistema: Categoría, Producto, Usuario, Pedido y DetallePedido. Todas heredan de la clase abstracta Base, que contiene datos comunes como id, createdAt y eliminado.

También se usan interfaces. La más importante es Calculable, implementada por Pedido, ya que un pedido necesita calcular su total a partir de los subtotales de sus detalles.

Los datos se guardan en colecciones ArrayList, administradas desde los servicios. Para manejar errores se usó IllegalArgumentException en validaciones comunes y dos excepciones propias: EntidadNoEncontradaException y StockInvalidoException.

3. Diseño y arquitectura

El proyecto se organizó separando responsabilidades:

- entities: clases del modelo.
- enums: valores fijos como rol, estado y forma de pago.
- interfaces: contratos como Calculable e Identificable.
- service: lógica del sistema, validaciones y colecciones.
- ui: menú y lectura de datos por consola.
- exception: excepciones propias.
- util: clases de ayuda.

El Main solo inicia el sistema, crea los servicios, carga datos iniciales y abre el menú. La lógica principal queda en los servicios, no en el menú.

El menú se dividió por módulos: categorías, productos, usuarios y pedidos. Además, se creó InputHelper para validar entradas comunes, como números, textos, IDs y confirmaciones.

Una decisión importante fue validar antes de modificar los objetos guardados. Por ejemplo, si se intenta editar un usuario con un mail repetido, el sistema muestra el error y no cambia el usuario original.

También se decidió simplificar el manejo de excepciones para seguir el estilo trabajado en los TP. Las validaciones comunes usan IllegalArgumentException; las búsquedas que no encuentran una entidad usan EntidadNoEncontradaException; y los problemas de stock en pedidos usan StockInvalidoException.

Menú principal del sistema.

```
put - FoodStore (run)

run:

FOOD STORE
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 4

PEDIDOS
1. Listar todos los pedidos
2. Listar pedidos por usuario
3. Consultar pedido por id
4. Crear pedido
5. Actualizar pedido
6. Eliminar pedido
0. Volver
```

4. Modelo aplicado en el código

Las clases principales son Categoría, Producto, Usuario, Pedido y DetallePedido. Todas heredan de Base, que contiene los datos comunes. De esta forma no se repiten atributos como id, createdAt y eliminado en cada clase.

Las relaciones principales son:

- Categoría agrupa varios productos. Es una relación de agregación, porque los productos pueden existir como objetos independientes.
- Cada Producto pertenece a una Categoría.
- Un Usuario puede tener varios pedidos.
- Cada Pedido pertenece a un Usuario.
- Pedido compone uno o más DetallePedido. Es una relación de composición, porque los detalles forman parte del pedido.
- Cada DetallePedido referencia un Producto, manteniendo el producto asociado al detalle de la compra.

La clase Pedido implementa la interfaz Calculable, porque debe calcular el total del pedido sumando los subtotales de sus detalles.

La baja lógica se maneja con el atributo eliminado. Cuando una entidad se elimina, no se borra de la colección, sino que queda marcada como eliminada. Después, los listados muestran solamente los registros activos.

En los pedidos, los detalles forman parte del pedido. El producto queda referenciado en el detalle para mantener el historial de la compra, aunque luego el producto sea editado o eliminado lógicamente.

Listado de pedidos funcionando en consola.

```
PEDIDOS
1. Listar todos los pedidos
2. Listar pedidos por usuario
3. Consultar pedido por id
4. Crear pedido
5. Actualizar pedido
6. Eliminar pedido
0. Volver
Seleccione una opcion: 1

Pedidos disponibles:
Pedido 1 - fecha: 2026-06-21 - usuario 1: Dario Frison - estado: CONFIRMADO - forma de pago: TARJETA - total: $20500.00
Pedido 2 - fecha: 2026-06-21 - usuario 2: Maxi Agüero - estado: PENDIENTE - forma de pago: EFECTIVO - total: $16000.00
```

5. Funcionamiento del sistema

El sistema tiene un menú principal con cuatro módulos: categorías, productos, usuarios y pedidos.

En categorías se puede listar, crear, editar y eliminar. El nombre y la descripción son obligatorios, y no se permiten nombres repetidos.

En productos se cargan datos como nombre, descripción, precio, stock, imagen, disponibilidad y categoría. El precio y el stock no pueden ser negativos, y el producto debe estar asociado a una categoría activa.

En usuarios se cargan nombre, apellido, mail, celular, contraseña y rol. El mail es obligatorio, se guarda en minúsculas y no se puede repetir.

En pedidos se selecciona un usuario activo, una forma de pago y uno o más productos. Cada detalle tiene cantidad, producto y subtotal. La cantidad debe ser mayor a cero y no puede superar el stock.

El total se calcula con el método `calcularTotal()` de `Pedido`. Antes de guardar el pedido, se validan todos los detalles. Recién después se descuenta el stock y se guarda en memoria. Esto evita que queden cambios parciales si aparece un error.

Para los errores comunes, como campos vacíos, valores negativos, IDs inválidos, duplicados o mail con formato incorrecto, se usa `IllegalArgumentException`. Si se busca una entidad que no existe, se usa `EntidadNoEncontradaException`. Si la cantidad pedida supera el stock, se usa `StockInvalidoException`.

Todas las eliminaciones son lógicas. Los registros no se borran físicamente de las colecciones.

Creación de un pedido

```

PEDIDOS
1. Listar todos los pedidos
2. Listar pedidos por usuario
3. Consultar pedido por id
4. Crear pedido
5. Actualizar pedido
6. Eliminar pedido
0. Volver
Seleccione una opcion: 4

Usuarios disponibles:
Usuario 1 - Dario Frison - frison@gmail.com - rol: ADMIN
Usuario 2 - Maxi Agüero - maxi@gmail.com - rol: USUARIO
Ingrese el id del usuario: 1
Forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione una opcion: 3

Productos disponibles:
Producto 1 - Hamburguesa - $8500.00 - stock: 18 - categoria: Comidas
Producto 2 - Papas fritas - $3500.00 - stock: 14 - categoria: Comidas
Producto 3 - Pizza muzzarella - $12000.00 - stock: 9 - categoria: Comidas
Producto 4 - Agua mineral - $2000.00 - stock: 28 - categoria: Bebidas
Producto 6 - Flan casero - $4000.00 - stock: 8 - categoria: Postres
Ingrese el id del producto: 2
Cantidad: 2
El producto se agrego al pedido.
Desea agregar otro producto? (S/N): n

Resumen del pedido:
Detalle 5 - Papas fritas - cantidad: 2 - subtotal: $7000.00
Total: $7000.00
Desea guardar el pedido? (S/N): s
El pedido se creo con el id 3 y un total de $7000.00.

```

Validación de error de cantidad mayor al stock disponible.

```

Productos disponibles:
Producto 1 - Hamburguesa - $8500.00 - stock: 18 - categoria: Comidas
Producto 2 - Papas fritas - $3500.00 - stock: 12 - categoria: Comidas
Producto 3 - Pizza muzzarella - $12000.00 - stock: 9 - categoria: Comidas
Producto 4 - Agua mineral - $2000.00 - stock: 28 - categoria: Bebidas
Producto 6 - Flan casero - $4000.00 - stock: 8 - categoria: Postres
Ingrese el id del producto: 3
Cantidad: 999
La cantidad supera el stock disponible de 9.

```

6. Pruebas realizadas

Las pruebas se hicieron manualmente desde la consola, usando la carga inicial y datos creados durante la ejecución.

También se probó la compilación con:

java 21

```
ant -f C:\repos\TPI_Programacion_II -Dnb.internal.action.name=rebuild clean jar
Output - FoodStore (clean,jar)
deps-clean:
Updating property file: C:\repos\TPI_Programacion_II\build\build-clean.properties
Deleting directory C:\repos\TPI_Programacion_II\build
clean:
init:
deps-jar:
Created dir: C:\repos\TPI_Programacion_II\build
Updating property file: C:\repos\TPI_Programacion_II\build\build-jar.properties
Created dir: C:\repos\TPI_Programacion_II\build\classes
Created dir: C:\repos\TPI_Programacion_II\build\empty
Created dir: C:\repos\TPI_Programacion_II\build\generated-sources\ap-source-output
Compiling 27 source files to C:\repos\TPI_Programacion_II\build\classes
compile:
Created dir: C:\repos\TPI_Programacion_II\dist
Copying 1 file to C:\repos\TPI_Programacion_II\build
Nothing to copy.
Building jar: C:\repos\TPI_Programacion_II\dist\FoodStore.jar
To run this application from the command line without Ant, try:
C:\Program Files\Java\jdk-21.0.11\bin/java -jar "C:\repos\TPI_Programacion_II\dist\FoodStore.jar"
deploy:
jar:
BUILD SUCCESSFUL (total time: 1 second)
```

Caso	Acción probada	Resultado esperado
<i>Menú</i>	Texto u opción fuera de rango	Muestra error y vuelve a pedir el dato
<i>Categoría</i>	Nombre repetido	No guarda la categoría
<i>Producto</i>	Precio o stock negativo	Rechaza el valor
<i>Usuario</i>	Mail repetido	No guarda el usuario
<i>Pedido</i>	Cantidad cero o mayor al stock	No agrega el detalle inválido
<i>Total</i>	Pedido con varios productos	Calcula el total correctamente
<i>Stock</i>	Guardar un pedido	Descuenta el stock
<i>Búsqueda</i>	ID inexistente	Informa que la entidad no fue encontrada
<i>Baja lógica</i>	Eliminar registros	No aparecen en listados, pero no se borran

También se revisó el CRUD completo de categorías, productos, usuarios y pedidos, junto con la actualización de estado y forma de pago.

7. Dificultades y soluciones

Una dificultad fue mantener sincronizadas algunas relaciones, como categoría-producto y usuario-pedido. Para resolverlo, los métodos de asociación actualizan la relación cuando corresponde y evitan duplicados.

Otra dificultad fue la edición de datos. Si se modificaba una entidad antes de validar todo, podía quedar información incorrecta. Por eso, primero se validan los datos nuevos y después se actualiza el objeto.

También hubo que cuidar el manejo del stock. La solución fue validar todo el pedido antes de descontar stock y guardarlo. Así no quedan descuentos parciales si un detalle tiene error.

Durante el desarrollo también se ajustó el manejo de excepciones y las validaciones comunes quedaron con `IllegalArgumentException`.

Por último, el menú se separó por módulos para que no quedara todo junto en una sola clase.

8. Bibliografía y enlaces

- Oracle. *Java Platform, Standard Edition 21 API Specification*.
<https://docs.oracle.com/en/java/javase/21/docs/api/>
- Oracle. *The Java Tutorials: Collections*.
<https://docs.oracle.com/javase/tutorial/collections/>
- Oracle. *The Java Tutorials: Exceptions*.
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- Repositorio: https://github.com/dfrison12/TPI_Prog_II
- Video demostrativo: <https://www.youtube.com/watch?v=1HPOnDzVsns>